

计算机系统基础实验

PA0

姓名: 伍知行 学号: 251220081

September 3, 2026

1 实验进度

我完成的实验内容如下:

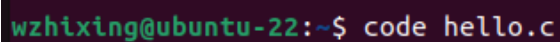
1. 安装 Ubuntu 22.04(通过 VirtualBox 虚拟机方式);
2. 下载必要的软件/开发工具;
3. 尝试在 Linux 环境下使用 gcc, gdb 等工具;
4. 获取 PA 的源代码.

2 必答题

I. 在 Linux 系统下写一个“Hello World”程序:

利用 VS Code 完成之:

- 1) 首先通过终端打开 VS Code:



```
wzhixing@ubuntu-22:~$ code hello.c
```

- 2) 利用 VS Code 写一个“Hello World”程序:

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
}
```

- 3) 利用命令行编译、运行之:

```
wzhixing@ubuntu-22:~$ gcc -O2 hello.c -o hello
wzhixing@ubuntu-22:~$ ./hello
Hello, World!
```

II. 写一个 Makefile 来编译 “Hello World” 程序:

利用 VS Code 创建并编写一个 Makefile 文件:

```
wzhixing@ubuntu-22:~/Programming$ code Makefile

CC = gcc
CFLAGS = -Wall -g
TARGET = hello
SRCS = hello.c

$(TARGET): $(SRCS)
    $(CC) $(CFLAGS) -o $(TARGET) $(SRCS)

clean:
    rm -f $(TARGET)

run: $(TARGET)
    ./${TARGET}

.PHONY: clean run
```

接下来, 先清除原先的可执行文件 hello 后, 再次编译、运行 “Hello World” 程序:

```
wzhixing@ubuntu-22:~/Programming$ make clean
rm -f hello
wzhixing@ubuntu-22:~/Programming$ ls
hello.c  Makefile
wzhixing@ubuntu-22:~/Programming$ make run
gcc -Wall -g -o hello hello.c
./hello
Hello, World!
```

III. 使用 GDB:

先编写一个快速排序算法:

```

#include <stdio.h>

void swap(int* a, int* b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

int partition(int* p, int len) {
    int i = -1, f = p[len - 1];
    for (int j = 0; j < len - 1; ++j) {
        if (p[j] <= f) i++, swap(p + i, p + j);
    }
    swap(p + i + 1, p + len - 1);
    return i + 1;
}

void quicksort(int* a, int len) {
    if (len == 0) return;
    int q = partition(a, len);
    quicksort(a, q);
    quicksort(a + q + 1, len - q - 1);
}

int main() {
    int a[15] = {14, 12, 10, 9, 3, 1, 6, 5, 8, 15, 2, 7, 4, 11, 13};
    quicksort(a, 15);
    for (int i = 0; i < 15; ++i) printf("%d ", *(a + i));
    printf("\n");
}

```

使用 GDB 调试:

- b: 设置断点

```

(gdb) b main
Breakpoint 1 at 0x10a0: file quicksort.c, line 25.
(gdb) b quicksort
Breakpoint 2 at 0x1320: quicksort. (9 locations)
(gdb) b partition
Breakpoint 3 at 0x12a0: file quicksort.c, line 9.

```

- r: 运行程序

```

(gdb) r
Starting program: /home/wzhixing/Programming/quicksort
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at quicksort.c:25
25      int main() {

```

- p: 打印此时程序中的变量/表达式的值

bt: 显示此时的调用栈情况

```
(gdb) p a[0]@15
$15 = {1, 2, 3, 4, 6, 5, 8, 10, 7, 9, 11, 12, 13, 15, 14}
(gdb) bt
#0  0x00005555555553cd in quicksort (len=1, a=0x7fffffffdea0) at quicksort.c:19
#1  quicksort (len=3, a=0x7fffffffdea0) at quicksort.c:21
#2  quicksort (len=10, a=0x7fffffffdea0) at quicksort.c:21
#3  quicksort (len=12, a=0x7fffffffdea0) at quicksort.c:21
#4  quicksort (a=<optimized out>, len=<optimized out>) at quicksort.c:21
#5  0x0000555555555146 in main () at quicksort.c:27
```

- n: 单步执行一行

```
(gdb) n
20      int q = partition(a, len);
```

- c: 在断点处中断后继续运行至下一个断点

```
(gdb) c
Continuing.

Breakpoint 2, quicksort (len=0, a=0x7fffffffdded4) at quicksort.c:19
19      if (len == 0) return;
```

- 最终程序运行结果:

```
main () at quicksort.c:28
28      for (int i = 0; i < 15; ++i) printf("%d ", *(a + i));
(gdb)
29      printf("\n");
(gdb)
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
```

IV. 在 Github 上添加 SSH key

我利用的是我自己 Github 账户已有的 SSH key:

- 1) 在虚拟机下创建.ssh 目录, 之后更改目录的权限。

```
wzhixing@ubuntu-22:~$ mkdir -p ~/.ssh
wzhixing@ubuntu-22:~$ chmod 700 ~/.ssh
```

- 2) 将已有的相应公钥、私钥复制至虚拟机中, 并更改私钥的权限:

```
wzhixing@ubuntu-22:~/.ssh$ ls -l
总计 16
-rw----- 1 wzhixing wzhixing 411 900  3 16:34 id_ed25519
-rw-r--r-- 1 wzhixing wzhixing  99 900  3 16:16 id_ed25519.pub
-rw----- 1 wzhixing wzhixing 978 900  3 16:36 known_hosts
```

```
wzhixing@ubuntu-22:~/.ssh$ chmod 600 ~/.ssh/id_ed25519
```

- 3) 最终与 Github 连接:

```
wzhixing@ubuntu-22:~/.ssh$ ssh -T git@github.com
Hi wzhixing1113! You've successfully authenticated, but GitHub does not provide shell access.
```

V. 对“好的提问”及“通过 STFW 和 RTFM 独立解决问题”的看法：

一、好的提问

根据我目前对“好的提问”的了解，好的问题应该具有如下特征：

- 好问题应该是基于一定的摸索和理解后提出的。
- 好问题应该尽可能的陈述清楚自己目前遇到的麻烦及曾做出过的努力。
- 好问题应该明确表达自己的需求，并且态度尽可能诚恳。

反观题目中给出的一系列问题，这些问题大多数都只有简短的描述，并且绝大部分可以通过阅读 manual 和上网查找来寻求相应的解答，因为这些通用和常见问题不可能在历史上从未出现过；

同时，题目中的一些问题诸如“段错误”、grep、curl 和 C 语言语法等问题，充分地反映出提问者对一些基本知识的了解完全不充分，是没有经历思考的体现；

因此，这些问题不是好问题，而是“蠢问题”。

二、通过 STFW 和 RTFM 独立解决问题

STFW 和 RTFM 都是“自我探索”的好途径好手段，它们都能降低我们问出“蠢问题”的概率。

原因很简单，“蠢问题”大概率是共性问题，当你问同伴/助教/老师一个能够通过 RTFM 或是 STFW 来获得解答的问题时，他们的回答大概率就是让你上网搜索/查询手册。到头来，还是得回去 RTFM 和 STFW，何苦这样浪费自己和别人的时间呢。

另一方面，就像上一部分所述，如果你问出了一个能够独立查询资料而解决掉的问题，那么就反映出你在这个问题上没有做出任何的努力或探索，因而提出了一个“蠢问题”。因此，学会通过 STFW 和 RTFM 来解决问题是至关重要的。

三、总结

就像题目中说的那样，成为一名合格的程序员就必定要懂得提问的智慧。提问的智慧不仅仅在于问题本身的内容，更在于你自身针对这个问题是否做出过一定的探索，是否对其有一定程度的理解。

掌握提问的智慧不是不问问题，而是不问“蠢问题”，不问能够通过简单努力、尝试、摸索就能获取解答的问题。因此，在问出一个问题前，我们应当反思：我们是否做出了相应的探索和努力，是否详细描述出了自己的需求和难处，是否表明自己的态度和意愿。如此，我们才能真正掌握提问的智慧。

3 实验心得和想法

本次 PA0 实验过程中，我对 Linux 操作系统有了初步的了解，包括整个操作系统的基本使用方法，开发环境配置以及对 Linux 和 Windows 系统的差异性的认识。更进一步的，我认为 PA0 实验对整个 PA 实验起到了较好的铺垫效果，为之后的 PA 实验开展打下了基础，是非常重要的一个阶段。